

INTERACTIVE STORYTELLER

Generative AI Project Documentation

GenAI Assessment Framework 2026

Project Category: Conversational AI & Chatbots

Project Type: Interactive Storyteller

1. Project Overview

Interactive Storyteller is a Generative AI-powered conversational application that allows users to create and continue dynamic stories through natural-language interaction. Instead of presenting a fixed story, the application uses an LLM to generate story content based on user input and maintains story-specific conversation history and state so the experience can continue across interactions.

The project was developed as part of the GenAI Assessment Framework 2026 under the Conversational AI & Chatbots category. It demonstrates practical skills in prompt engineering, conversational context management, backend API development, frontend development, database persistence, testing, version control, containerization, and cloud deployment.

2. Problem Statement

Traditional digital storytelling applications generally provide predetermined content and offer limited user influence over how a story develops. Users may read a story, but they often cannot meaningfully shape its direction through an ongoing conversation.

The Interactive Storyteller addresses this problem by combining conversational interaction with Generative AI. Users can provide story input and interact with generated narrative content, while the application maintains relevant conversation and story state.

3. Project Objectives

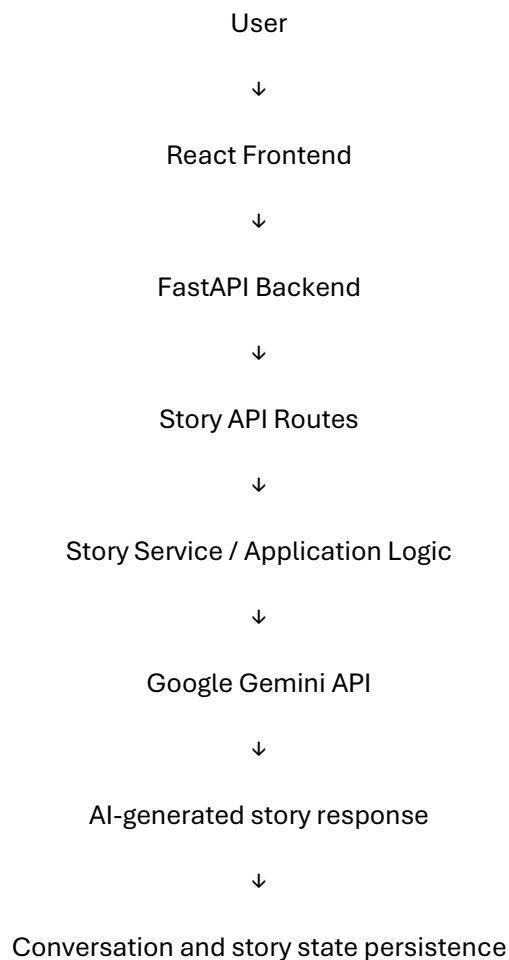
- Build a functional Generative AI conversational application from concept through deployment.
- Integrate a Google Gemini-based LLM into a Python backend.
- Use prompt engineering to guide AI-generated storytelling responses.
- Maintain conversation history and story state for individual story sessions.
- Provide an interactive React-based user interface.
- Persist story data using SQLite and SQLAlchemy.
- Expose backend functionality through FastAPI REST endpoints.
- Provide API documentation through FastAPI Swagger/OpenAPI.
- Dockerize the backend application.
- Deploy the backend and frontend to public cloud platforms.
- Use Git and GitHub for version control and project delivery.
- Document the project professionally for assessment and review.

4. Key Features

- AI-powered dynamic story generation.
- Conversational storytelling based on user input.
- Interactive story progression and choices.
- Story-specific conversation history.
- Persistent story state.
- Multiple story sessions.
- Story history listing.
- Story retrieval and deletion functionality.
- Loading indicators and user-facing error handling.
- REST API backend using FastAPI.
- SQLite database persistence using SQLAlchemy.
- Swagger/OpenAPI API documentation.
- Dockerized backend deployment.
- Publicly deployed frontend and backend.

5. System Architecture

The application follows a frontend-backend-service-database architecture with an external LLM API integration.





SQLite Database

The frontend communicates with the deployed FastAPI backend through HTTP requests. The backend handles story operations, invokes the LLM when AI-generated content is required, and persists story state and conversation history in the database.

6. Technology Stack

Layer	Technology	Purpose
Programming Language	Python 3.12+	Backend development
Backend Framework	FastAPI	REST API and application server
LLM	Google Gemini API	Generative AI story creation
Frontend	React	Interactive user interface
Frontend Tooling	Vite	Frontend development and build tooling
Database	SQLite	Persistent database storage
ORM	SQLAlchemy	Database access and persistence
API Documentation	Swagger / OpenAPI	Interactive API documentation
Containerization	Docker	Backend containerization
Version Control	Git	Source code version control
Repository	GitHub	Remote source code hosting
Backend Deployment	Render	Public backend hosting
Frontend Deployment	Vercel	Public frontend hosting

7. Project Structure

The project is organized into separate backend, frontend, testing, and documentation areas.

- app/ – FastAPI backend application.
- app/api/ – API route definitions.
- app/database/ – Database configuration and models.
- app/models/ – Application data models.
- app/prompts/ – Prompt-related project files.
- app/services/ – Story and database service logic.
- frontend/ – React/Vite frontend application.
- tests/ – Project test files.
- documentation/ – Project documentation and screenshots.
- Dockerfile – Backend Docker image configuration.
- requirements.txt – Python dependencies.
- .env – Local environment configuration and API secrets.
- .env.example – Example environment variable configuration.
- .gitignore – Files excluded from version control.
- README.md – Project overview, setup, usage, deployment, and screenshots.

8. Prompt Engineering and LLM Integration

The application uses a Google Gemini model to generate story content. Prompt engineering is used to guide the model toward the intended storytelling behavior and to provide the conversational context required for continuity.

Prompt Engineering Approach

- The application supplies the model with relevant user interaction and story context.
- The prompt structure is designed to encourage coherent continuation of the current story.
- AI responses are generated dynamically rather than using fixed story text.
- Conversation history is used to maintain continuity across interactions.
- Prompt-related project files are separated from application logic to keep the codebase organized.

Prompt design is a core part of the application's behavior because the quality of generated stories depends on how the model is instructed and what contextual information is provided.

9. Conversation History and Memory Management

Each story session is associated with a unique story identifier. The application persists the story's state and conversation history so that a session can be retrieved and continued.

Stored Information

- Story ID – uniquely identifies a story session.
- State – stores the current story state as structured JSON data.
- Conversation – stores the sequence of conversational messages as JSON data.

Conversation Flow

1. The user starts or selects a story session.
2. The frontend sends the user interaction to the backend.
3. The backend retrieves the relevant story state and conversation history.
4. The application invokes the LLM with the required context.
5. The AI generates the next story response.
6. The updated state and conversation are persisted.
7. The frontend displays the new response and available interaction options.

10. Backend API

The backend is implemented using FastAPI. API routes provide the operations required by the frontend for creating, retrieving, listing, updating, and deleting story sessions.

Operation	Purpose
POST /story	Create or process a story interaction.
GET /stories	Retrieve available story sessions for the history view.
GET /story/{story_id}	Retrieve a specific story session.
DELETE /story/{story_id}	Delete a specific story session.

The exact request and response schemas can be inspected using the application's interactive Swagger/OpenAPI documentation available at the deployed backend's /docs endpoint.

11. Frontend

The frontend is implemented using React and provides the user-facing storytelling experience. It communicates with the deployed FastAPI backend through HTTP requests.

- Main storytelling interface for entering user input.
- Display of AI-generated story responses.
- Interactive choices that allow the user to influence the story.
- Story history interface for viewing existing sessions.
- Story selection and continuation.
- Story deletion functionality.
- Loading indicators during asynchronous operations.
- Error messages when requests fail.

12. Database and Persistence

SQLite is used as the project's database, with SQLAlchemy providing ORM-based database access. The database layer includes session management and service functions for story creation, retrieval, listing, updating, and deletion.

The story data model stores the story identifier together with serialized story state and conversation history. This allows each story session to maintain its own context and continue independently from other sessions.

13. Testing and Validation

The project was tested progressively during development and deployment.

- Database functionality testing.
- Gemini/LLM integration testing.
- Backend API testing through FastAPI Swagger/OpenAPI documentation.
- Testing of story creation and interaction flows.
- Testing of story history retrieval.
- Testing of story deletion.
- Testing of Dockerized backend execution.
- Testing of the deployed Render backend.
- Testing of the deployed Vercel frontend.
- Testing of frontend-to-backend communication.
- Testing and resolution of CORS configuration issues.
- Verification of the final public application.

The final deployed application was tested through the public frontend and backend endpoints, and the application was confirmed to be working after resolving production API URL and CORS configuration issues.

14. Dockerization

The backend was containerized using Docker. The Docker configuration packages the Python application and its dependencies into a reproducible runtime environment.

- Dockerfile created for the backend.
- Python dependencies declared in requirements.txt.
- .dockerignore used to exclude unnecessary local files.
- Environment variables supplied at runtime rather than hardcoded into the image.
- Docker container tested locally before deployment.

15. Deployment

The application was deployed as separate frontend and backend services.

Component	Platform	URL
Frontend	Vercel	https://interactive-storyteller-drab.vercel.app/
Backend	Render	https://interactive-storyteller-rjm8.onrender.com
API Documentation	Render / Swagger	https://interactive-storyteller-rjm8.onrender.com/docs

The frontend was configured to communicate with the deployed Render backend rather than the local development address. The production deployment was verified after updating the frontend API configuration and resolving the required CORS configuration.

16. Environment Variables and Security

API credentials and environment-specific configuration are managed using environment variables. Sensitive values are stored in the local .env file and are not intended to be committed to the Git repository.

- API keys are not hardcoded in application source code.
- .env is excluded from version control.
- .env.example provides a safe template for required configuration.
- Deployment environments are configured with their required environment variables.

17. Git and Version Control

Git and GitHub were used throughout development. The repository contains meaningful milestone commits covering the project's development lifecycle.

- Initial project setup.
- Backend implementation.
- Frontend implementation.
- Dockerization.
- Deployment and project documentation.
- Frontend connection to the deployed backend.
- Screenshot and README documentation updates.

The final working tree was verified as clean after the completed documentation and screenshot updates.

18. Challenges and Solutions

Challenge	Solution
Docker build initially failed because the Dockerfile was empty.	Created and configured the backend Dockerfile.
Docker container failed because SQLAlchemy was missing.	Updated requirements.txt to include the required database dependency and rebuilt the image.
Production frontend initially attempted to call localhost.	Updated the frontend API base URL to use the deployed Render backend.
Production requests were blocked by CORS.	Configured FastAPI CORS middleware to allow the deployed Vercel frontend origin.
Deployment and API availability were intermittently affected during cloud startup.	Verified deployment status, redeployed when necessary, and retested the public API.
Git repository needed updated deployment/documentation changes.	Committed and pushed the final changes to GitHub with milestone commit messages.

19. Screenshots

The repository currently includes the following screenshots in documentation/screenshots/:

- main-interface.png – Main application interface.
- deployed-application.png – Publicly deployed application.
- api-documentation.png – FastAPI Swagger/OpenAPI documentation.

These screenshots are also referenced from README.md.

20. README and User Guide

The README.md file provides the project's main documentation entry point. It includes the project overview, features, technology information, setup and usage information, deployment details, links, and screenshots.

The README is intended to allow a reviewer or developer to understand the project, run the application locally, access the deployed application, and inspect the API documentation.

21. Future Enhancements

- Migrate from SQLite to PostgreSQL or another production-oriented database.
- Add user authentication and account-based story ownership.
- Improve long-term story memory with structured character and world state.
- Add streaming AI responses for a more responsive user experience.
- Add richer branching story structures and more advanced choice management.
- Add automated unit and integration test coverage for critical application logic.
- Add observability and production monitoring for errors and performance.
- Add optional image generation for visual story scenes.
- Add voice input and text-to-speech capabilities.

22. Assessment Deliverable Mapping

Assessment Requirement	Project Evidence	Status
------------------------	------------------	--------

GitHub Repository	GitHub repository containing source code and project history	Completed
Source Code	Backend, frontend, database, services, tests, and configuration	Completed
README.md	Project documentation with setup, usage, deployment, and screenshots	Completed
Screenshots	Three screenshots stored in documentation/screenshots/	Completed
Documentation	This final project documentation document	Completed
Working Demo	Public Vercel frontend connected to Render backend	Completed
Deployment Link	Public frontend and backend URLs	Completed
Environment Variables	.env configuration and .gitignore protection	Completed
Dockerization	Dockerfile and Docker-tested backend	Completed
API Documentation	FastAPI Swagger/OpenAPI documentation	Completed

23. Conclusion

The Interactive Storyteller project demonstrates the end-to-end development of a Generative AI conversational application. The project combines an LLM-powered storytelling experience with a React frontend, FastAPI backend, persistent database storage, API documentation, Docker containerization, Git/GitHub version control, and cloud deployment.

The completed solution demonstrates practical understanding of Generative AI application development, prompt-driven conversational behavior, context and memory management, software engineering practices, testing, documentation, and deployment. The project is publicly accessible and supported by a GitHub repository, README, screenshots, API documentation, and this final project documentation.

24. Project Links

- Live Application: <https://interactive-storyteller-drab.vercel.app/>
- Backend: <https://interactive-storyteller-rjm8.onrender.com>
- API Documentation: <https://interactive-storyteller-rjm8.onrender.com/docs>